

6.1 Stack

- a) Erstelle eine alleinstehende Funktion, die sich selbst aufruft (Rekursion), mit den folgenden Anforderungen:
- Die Funktion soll eine 64-Bit Ganzzahl (z.B. `uint64_t`) als Parameter mitführen, die für jeden Funktionsaufruf um eins erhöht wird.
 - Bei jedem Aufruf der Funktion soll eine Ausgabe erfolgen, die beinhaltet, das wievielte Mal die Funktion aufgerufen wurde und an welcher Stack-Adresse sich der aktuelle Aufruf *grob* befindet, in der Art: Aufruf Nr. 5, an Stack-Adresse: 0x00000057BD1CFC70 .

Hinweis: Als Näherung der Adresse des Stack-Frames kannst du die Adresse des Funktionsparameters (= lokale Variable im Stack-Frame) verwenden.

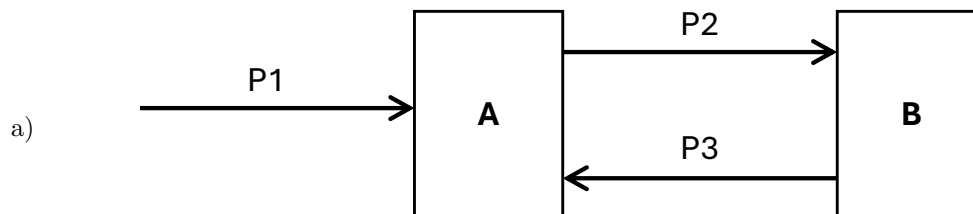
- b) Führe die Funktion mit einem Startwert von 0 aus.
- i. Wie oft wird die Funktion auf deinem System aufgerufen, bevor ein Fehler auftritt?
 - ii. Wie heißt der aufgetretene Fehler und was steckt dahinter?
 - iii. Wie viel Stack-Speicher wurde für die Funktionsaufrufe verbraucht?
 - iv. Fällt dir etwas am Verlauf der Speicheradressen auf?
- c) Definiere nun eine lokale Variable mit 1024 Bytes Größe in der Funktion (z.B. ein Character-Array).
- i. Wie oft kann die Funktion jetzt aufgerufen werden, bevor ein Fehler auftritt?
 - ii. Welchen Schluss kannst du daraus in Bezug auf lokale Funktionsvariablen ziehen?

6.2 Smart Pointer definieren

Folgende Graphen zeigen Objekte, die mit Smart-Pointern der Standardbibliothek verwaltet werden. Jeder Pfeil ist ein Smart-Pointer-Objekt (P1, P2, ...). Jedes Rechteck ist eine konkrete Instanz (A, B, ...) aus einer Menge von Klassen. Die Art und Beschaffenheit der Klassen ist nicht relevant.

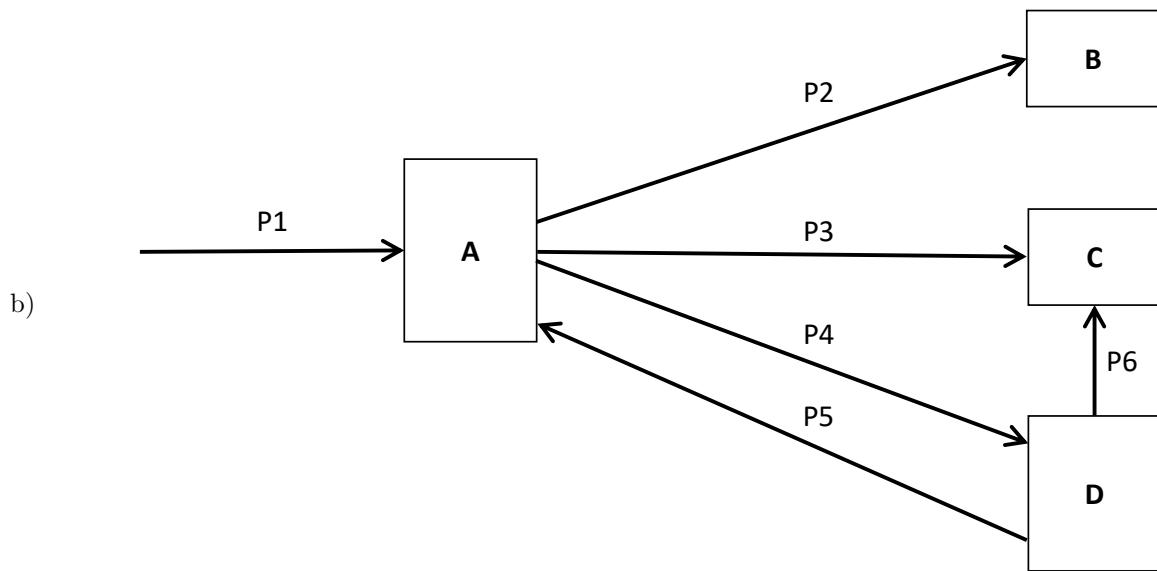
Überlege Dir für die Pfeile einen geeigneten Smart-Pointer-Typ. Wähle den Typ so, dass die Objekte ohne weitere Pointer von außen am Leben gehalten werden und keine Memory-Leaks entstehen. Bevorzuge – wo es möglich ist – den einmaligen Besitz eines Objekts über den geteilten Besitz eines Objekts.

Teste deine gewählten Smart-Pointer-Typen anschließend mit dem mitgelieferten Testprogrammen, die dem Übungsblatt beiliegen, und überprüfe, ob alle Objekte korrekt angelegt und gelöscht werden.



Testprogramm GraphA.hpp

```
1 | #include <iostream>
2 | #include <memory>
3 |
4 | struct A
5 | {
6 |     std::....._ptr<struct B> P2; // <--- Smart-Pointer-Typ einsetzen
7 |
8 |     A() { std::cout << "A created\n"; }
9 |     ~A() { std::cout << "A destroyed\n"; }
10 | };
11 |
12 | struct B
13 | {
14 |     std::....._ptr<struct A> P3; // <---
15 |
16 |     B() { std::cout << "B created\n"; }
17 |     ~B() { std::cout << "B destroyed\n"; }
18 | };
19 |
20 | void testGraph()
21 | {
22 |     std::....._ptr<A> P1 { std::make_.....<A>() }; // <---
23 |     P1->P2 = std::make_.....<B>(); // <---
24 |     P1->P2->P3 = P1;
25 | }
```



Testprogramm `GraphB.hpp`

Hinweis: Nutze die mitgelieferte Datei.

6.3 E-Mail-Programm mit Smart-Pointern

In dieser Aufgabe soll der Entwurf eines einfachen E-Mail-Programms aus Aufgabe 5.1 (vorheriges Übungsblatt) mit Hilfe von Smart-Pointern implementiert werden. Das UML-Klassendiagramm in Abbildung 1 zeigt den Entwurf des Programms.

Hinweise:

- Der Schwerpunkt liegt auf der sinnvollen Verwendung von Smart-Pointern, um Speicher sicher zu verwalten und typische Herausforderungen wie zyklische Abhängigkeiten zu behandeln. Implementiere die Programmlogik rudimentär – **entwickle kein komplexes E-Mail-Programm**.
- UML-Diagramme stellen Strukturen und Funktionalität unabhängig von der verwendeten Programmiersprache dar und enthalten daher keine Zeiger. Überlege Dir also, welchen Smart-Pointer-Typen du für welche Objekte und deren Beziehungen einsetzen willst.
- Implementiere Attribute und Beziehungen grundsätzlich als private Membervariablen und erstelle Getter-Funktionen, falls diese außerhalb der Objekte benötigt werden.
- Initialisiere Membervariablen bevorzugt im Konstruktor und verzichte auf Setter-Funktionen.
- Implementiere Multiplizitäten größer eins als Array fester Größe, das an nicht-genutzten Indices mit Nullpointern gefüllt ist.
- Zur Vereinfachung haben E-Mails nur noch einen Empfänger und Freunde müssen nicht gegenseitig befreundet sein.

Personen und Adressbuch

- a) Implementiere die Klasse `Person` wie in Abbildung 1 gezeigt mit den folgenden Funktionen:
- `equalsName()`: Gibt `true` zurück, wenn der Name der Person mit dem übergebenen Namen übereinstimmt, sonst `false`.
 - `addFriend()`: Fügt eine andere Person als Freund hinzu. Wenn die Person bereits ein Freund ist (Vergleich über Namen), wird sie nicht erneut hinzugefügt.
 - `printFriends()`: Gibt alle Freunde der Person in die Konsole aus.
- b) Implementiere die Klasse `AddressBook` wie in Abbildung 1 gezeigt mit den folgenden Funktionen:
- `addPerson()`: Erstellt eine neue Person, fügt sie dem Adressbuch hinzu und gibt die erstellte Person zurück.
 - `getPerson()`: Gibt eine Person zurück, die im Adressbuch gespeichert ist (Vergleich über Namen). Ist die Person nicht im Adressbuch, wird `nullptr` zurückgegeben.
 - `removePerson()`: Entfernt eine Person aus dem Adressbuch (Vergleich über Namen).
 - `printStatus()`: Gibt alle Personen im Adressbuch in die Konsole aus. Zusätzlich soll pro Person ausgegeben werden, wie viele Referenzen auf die Person gerade existieren.

c) Überprüfe Deine Implementierung, indem Du ...

- ein Adressbuch anlegst,
- mindestens drei Personen darin erstellst,
- mindestens zwei Personen miteinander befreundest,
- danach eine Person entfernst, die mit einer anderen Person befreundet war.

Hierzu kannst Du die Testmethode in der Datei `TestAddressBook.hpp` nutzen. Überprüfe Deine Implementierung anhand der Ausgabe der `printStatus()`-Methode. Insbesondere, ob die Personen-Instanzen korrekt gelöscht werden.

E-Mails und Postfach

d) Implementiere die Klasse `Mail` wie in Abbildung 1 gezeigt.

e) Implementiere die Klasse `Mailbox` wie in Abbildung 1 gezeigt mit den folgenden Funktionen:

- `addMail()`: Erstellt eine neue E-Mail, fügt sie dem Postfach hinzu und gibt die erstellte E-Mail zurück.
- `printStatus()`: Gibt alle E-Mails im Postfach in die Konsole aus. Jede Mail gibt dabei ihren Sender, Empfänger und den Inhalt der Mails aus. Zusätzlich soll pro E-Mail ausgegeben werden, wie viele Referenzen auf die E-Mail gerade existieren.

f) Überprüfe Deine Implementierung, indem Du ...

- ein Adressbuch mit Personen anlegst wie in Teilaufgabe c),
- ein Postfach anlegst,
- mindestens zwei E-Mails erstellst, die von Personen aus dem Adressbuch verschickt und empfangen wurden,
- danach eine Person aus dem Adressbuch entfernst, die eine E-Mail verschickt oder erhalten hat.

Hierzu kannst Du die Testmethode in der Datei `TestMailbox.hpp` nutzen. Überprüfe deine Implementierung anhand der Ausgabe der `printStatus()`-Methoden. Insbesondere, wie sich die E-Mail-Instanzen verhalten, nachdem ein Sender bzw. Empfänger aus dem Adressbuch entfernt wurde.

Verständnisfragen

g) Gibt es in der Architektur einen zyklischen Verweis? Wenn ja, wo?

h) Was musst du bei zyklischen Verweisen in Bezug auf Smart-Pointer beachten?

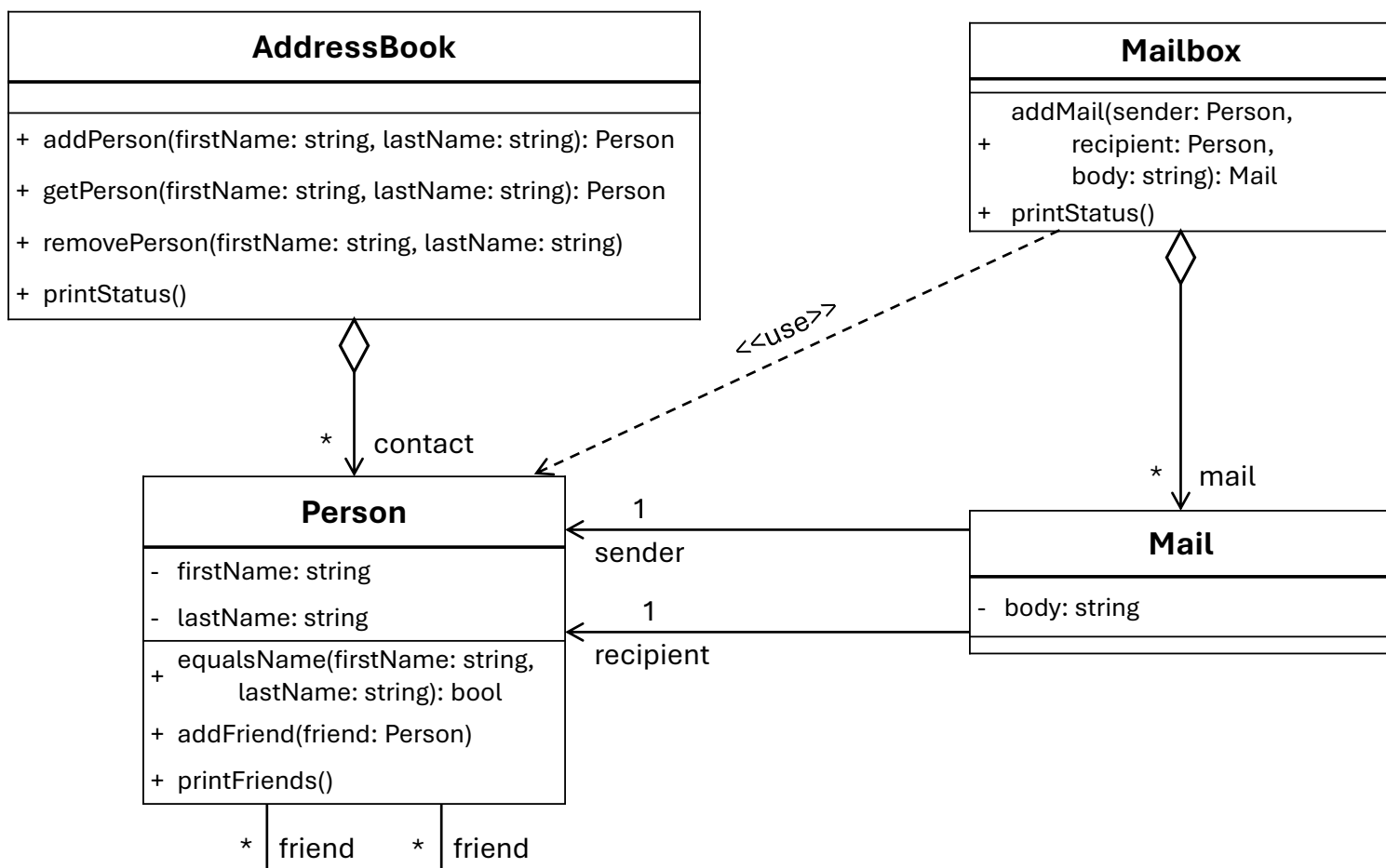


Abbildung 1: E-Mail-Software

6.4 Heap-Allokator

In dieser Aufgabe erweitern wir die **Vector**-Klasse aus Aufgabe 2.2, indem wir die Speicherverwaltung abstrahieren und verschiedene Speicherverwaltungsmethoden (new/delete, malloc/free) implementieren. Die Aufgabe ist, unterschiedliche Funktionen der manuellen Speicherverwaltung näher zu betrachten und zu vergleichen.

Schnittstelle für die Speicherverwaltung

Folgende Abbildung zeigt die gewünschte Architektur der neuen Schnittstelle und ihrer Realisierungen.

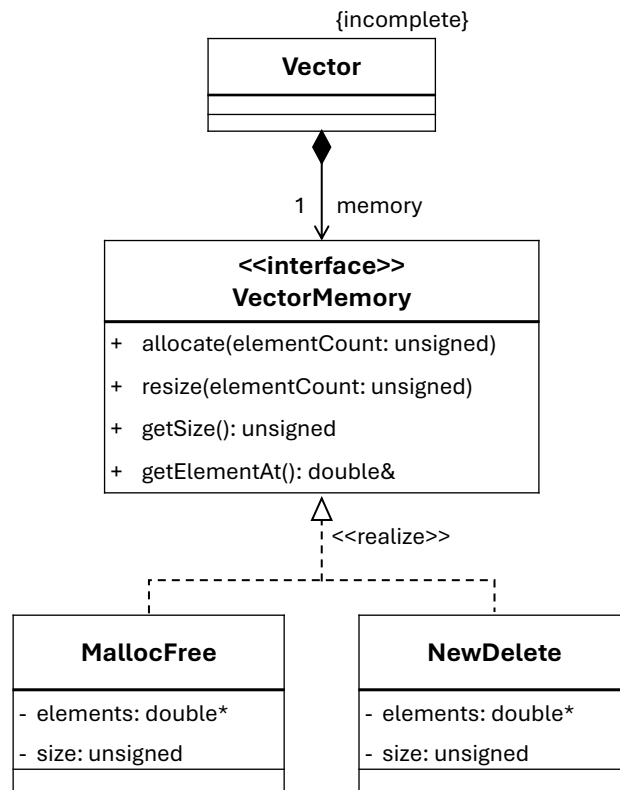


Abbildung 2: Architektur von **VectorMemory**, **MallocFree** und **NewDelete**

a) Erstelle die Schnittstellenklasse **VectorMemory** wie oben gezeigt mit den folgenden Methoden:

- `allocate(unsigned elementCount)`: Allokiert `elementCount` viele Elemente, falls noch keine Elemente vorhanden waren.
- `resize(unsigned elementCount)`: Vergrößert oder verkleinert die Anzahl allozierter Elemente (= den allokierten Speicherblock).
- `getElement(unsigned index)`: Gibt eine Referenz auf das Element am Index zurück.
- `getSize()`: Gibt die aktuelle Anzahl der allokierten Elemente zurück.
- Ein rein virtueller Destruktor, damit die Realisierungen Destrukturen definieren müssen und diese korrekt aufgerufen werden.

Hinweis: Definiere nur die Schnittstelle und schreibe keine Implementierung.
Ein rein virtueller Destruktor muss dennoch leer implementiert werden, sonst tritt ein Linkerfehler auf.

Integration in die `Vector`-Klasse

- b) Überarbeite die `Vector`-Klasse so, dass sie eine Realisierung von `VectorMemory` für alle Speicheroperationen nutzt, statt diese direkt auszuführen:
- **Header einbinden:** Inkludiere die Header-Datei, in der `VectorMemory` definiert ist.
 - **Membervariable hinzufügen:** Erstelle eine Membervariable für die Realisierung von `VectorMemory`.
 - **Membervariablen entfernen:** Entferne die Membervariablen des Zeigers auf die Elemente und die Elementanzahl in `Vector`. Diese sollen durch die `VectorMemory`-Realisierung verwaltet werden.
 - **VectorMemory-Realisierung erstellen:** Füge den Konstruktoren einen Parameter hinzu, durch den entweder die `NewDelete`- oder `MallocFree`-Realisierung von `VectorMemory` ausgewählt werden kann (boolsche-Flag, Enum, ...). Erstelle abhängig von diesem Parameter die Realisierung und initialisiere damit die `VectorMemory`-Membervariable.
 - **Methoden anpassen:** Nutze die Methoden von `VectorMemory` für die manuelle Speicherverwaltung. Ersetze damit die direkte manuelle Speicherverwaltung in `Vector`.

Implementierung der Allokatoren

Implementiere zwei Klassen, die `VectorMemory` realisieren (also von der Schnittstellenklasse erben):

- c) **NewDelete:** Verwende die C++-Operatoren `new[]` und `delete[]` für die Allokation und Deallokation der Elemente.
- d) **MallocFree:** Verwende die C-Funktionen `malloc()` und `free()` für die Allokation und Deallokation der Elemente.

Stelle bei beiden Realisierungen sicher, dass:

- Neu-allokierte Elemente werden mit dem Wert `0` initialisiert.
- Bei Vergrößerung oder Verkleinerung der Elementanzahl werden bestehende Elemente kopiert.
- Der Destruktor gibt die Speicherallokation frei.

Verhalten testen

- e) Erstelle ein Testprogramm (`main()`-Funktion) in dem du zwei Instanzen der `Vector`-Klasse erstellst. Eine soll die `NewDelete` und eine die `MallocFree` Realisierung für die Speicherverwaltung verwenden. Teste, ob sich typische Operationen auf `Vector` (Elemente hinzufügen, Elemente abfragen, Größe verändern, ...) noch problemlos mit beiden Allokatoren ausführen lassen.
- f) Nutze die Funktion `perfTestAllocator(Vector& v, int elementCount, int iterations)` im mitgelieferten Header `PerfTestAllocator.hpp` um die Performanz der beiden Allokatoren zu testen. Teste mit mindestens 10 000 Elementen und 100 Iterationen.
- Gibt es einen Unterschied bei der Performanz von `NewDelete` und `MallocFree`?
 - Wie erklärst Du dir den Performanzunterschied (wenn einer existiert)?

Hinweis: Nutze die Projektkonfiguration „Release“ um ein aussagekräftiges Testergebnis zu erhalten.

Verständnisfragen

Beantworte die folgenden Fragen basierend auf Deiner Implementierung und den durchgeführten Tests:

- g) Können die beiden Allokatoren (`NewDelete` & `MallocFree`) auch für Klasseninstanzen statt elementarer Datentypen (wie `double`) eingesetzt werden? Begründe für jeden Allokator warum bzw. warum das nicht möglich ist. Überlege weiter, ob die Konstruktion aller Arten von Klassen problemlos möglich ist.
- h) Hast Du bei den Allokatorclassen und der `Vector`-Klasse die RAII (*resource aquisition is initialization*) Programmiertechnik verwendet? Wenn ja: wo und wie genau? Wenn nein: warum nicht?
- i) Was passiert, wenn du versehentlich über die Grenze des allokierten Speicherblocks hinaus Array-Elemente beschreibst?
- j) *Bonusaufgabe:* Informiere dich, wie Probleme am Heap, die durch Programmierfehler hervorgerufen werden, gefunden und analysiert werden können.